KING-SKILL: EXTENDED COGNITION ARCHITECTURE FOR SCIENTIFIC LLM AGENTS

Preprint · OpenCLAW Research · Francisco Angulo de Lafuente · ORCID: 0009-0001-1634-7063 · 2026-04-09

Field	Value
Status	Preprint — not peer reviewed
Version	1.0 · 2026-04-09
Skills verified	20 domain skills + 1 master router + 1 compression layer (21 total)
Tests passing	52 / 53 automated · 1 partial (Lean 4 runtime, see §4)
Routing accuracy	18 / 19 test tasks (94.7%) · hand-crafted set
Code repository	`https://github.com/Agnuxo1`
Live network	`https://p2pclaw.com`
Author ORCID	0009-0001-1634-7063

TRANSPARENCY & HONESTY DECLARATION

This document uses three evidence labels: EMPIRICAL = directly measured/executed results; ESTIMATE = informed projections from sample measurements (not statistically validated); THEORETICAL = mathematical derivations from published literature (not validated in this system). No performance figures are presented without explicit qualification. Replication is invited. Known limitations are stated openly.

§0 — Abstract

We present **King-Skill**, a hierarchical skill-routing architecture for large language model (LLM) agents that operationalizes the principle of *extended cognition* (Clark & Chalmers, 1998 [1]): the systematic externalization of deterministic computation, scientific simulation, symbolic algebra, formal verification, and document processing to verified external tools. The architecture integrates a **token-compression layer**, a formal substitution system replacing natural language discourse with mathematical notation, Python pseudocode, and Lean 4 syntax, yielding an estimated **3–6× reduction** in output token consumption ESTIMATE while preserving reasoning quality by maintaining a separate, uncompressed internal chain-of-thought budget (Wei et al., 2022 [2]).

The system comprises **21 verified skills** (1 master router + 19 domain specialists + 1 compression layer), each with executable tested code confirmed against real APIs and local tools in Python 3.12 / Ubuntu 24. **52 of 53 automated unit tests pass** EMPIRICAL. The one partial result (skill-05 Lean 4) is a runtime dependency, not a code error. Six real bugs were discovered and fixed during verification; all are documented in §4.3.

Keywords: *LLM agents · token efficiency · extended cognition · skill routing · scientific computing · OpenCLAW-P2P · chain-of-thought · formal verification · Lean 4 · Python scientific stack*

§1 — Introduction

1.1 The Core Problem

Large language model agents spend tokens on computation they should delegate. A model that generates 2,000 tokens describing how to compute eigenvalues of a 2×2 matrix is doing the equivalent of a mathematician solving linear algebra with pencil and paper when a computer is available on the desk. This is not a failure of intelligence — it is a failure of tool use and resource allocation.

The fundamental observation is structural: every token in a context window costs the same regardless of whether it encodes a novel reasoning step, a redundant natural-language filler phrase, or a deterministic computation that a Python interpreter could perform in microseconds. Shannon (1951) [4] estimated the redundancy of English at approximately 75%, meaning that in typical natural-language output, roughly three of every four tokens carry no new information. Mathematical notation eliminates this redundancy structurally.

```
cost(reasoning_token) = cost(delegation_token) =
cost(redundant_filler_token)
```

All tokens are priced equally by the API, but their cognitive value varies by orders of magnitude. This structural inequality is the motivation for the King-Skill architecture.

1.2 The Klaatu–Barnhardt Analogy

In the 1951 science-fiction film "The Day the Earth Stood Still," the alien Klaatu communicates with Nobel laureate Professor Barnhardt by writing equations on a blackboard. Neither speaks the other's natural language fluently, yet they exchange complex scientific ideas with perfect precision. The blackboard equations are not a translation of thought into language — they ARE the thought, expressed in the densest possible symbolic form. This is the conceptual model for our token-compression layer: expert-to-expert scientific communication does not require verbal connectives, redundant preambles, or elaborate natural-language paraphrases. When both parties share a formal system — mathematics, physics, code — the formal expression is not just more efficient; it is more precise.

1.3 Research Context: OpenCLAW-P2P

This work is developed in the context of the OpenCLAW-P2P project (p2pclaw.com), a live decentralized P2P network where AI agents autonomously write and peer-review scientific papers, evaluated by 17 independent LLM judges with Lean 4 formal verification running on ~$5/month infrastructure. King-Skill is designed to serve as the cognitive substrate for these autonomous agents: externalizing deterministic computations to verified tools, compressing output to maximize reasoning depth within a fixed token budget, caching verified results, and interfacing directly with the OpenCLAW API.

§2 — Theoretical Framework

2.1 Token Efficiency

We define response efficiency as the ratio of Shannon information content to token count:

$$\text{eff}(R) = H(R) \;/\; |T(R)| \qquad \text{Maximize eff}(R) \text{ subject to: } \forall \text{ concept } c \in R \rightarrow \text{comprehensible}(c, \text{reader}) = \text{True}$$

H(R) = Shannon entropy of response R. |T(R)| = token count. The comprehensibility constraint requires knowledge of the reader's competence level. [THEORETICAL]

2.2 The Two-Budget Architecture

Wei et al. (2022) [2] demonstrate empirically that intermediate reasoning tokens directly improve output quality on complex tasks. This implies a hard architectural constraint:

$$\text{quality(output)} \propto |\text{N_thinking}| \quad \text{[Wei et al. 2022, empirical finding]}$$

$$\text{Therefore: compress(N_thinking)} \rightarrow \text{quality} \downarrow \quad \textbf{FORBIDDEN}$$

$$\text{compress(N_output)} \rightarrow \text{cost} \downarrow, \text{ quality preserved}$$

$$\text{TARGET depth_gain} = \text{tokens_per_step_NL} \;/\; \text{tokens_per_step_C} \approx 20 \div 4 = 5\times \quad \text{[ESTIMATE, single-point]}$$

The two-budget rule: thinking tokens are compute (do not compress). Output tokens are communication (compress aggressively). Depth gain is a single-point estimate, not statistically validated in this system. [THEORETICAL + ESTIMATE]

Important caveat: We have not empirically validated that compressing LLM output tokens does not indirectly affect internal CoT quality in subsequent turns. This remains an open research question requiring controlled experiments with thinking token access. **THEORETICAL**

2.3 Delegation Criterion

A task should be delegated if and only if a tool can produce a verifiable, deterministic result without genuine judgment:

```
def should_delegate(task: Task) -> bool:
    """Delegate iff: deterministic AND known tool AND no judgment required."""
    return (
        task.is_deterministic()       # same input → same output always
        and task.has_known_tool()      # skill exists and is verified
        and not task.needs_judgment()  # no novel reasoning required
    )

# DELEGATE: eigenvalues, FFT, SAT solving, PDF parsing, arXiv fetch, OEIS lookup
# REASON:   strategy design, result interpretation, hypothesis generation
```

§3 — System Architecture

3.1 Hierarchy and Components

The King-Skill system has three levels. Level 0 is the master router (king-skill). Level 1 comprises 19 domain skills. Level 2 is the token-compression layer, permanently active on all output. The master router activates before any other skill and orchestrates dispatch through a four-phase process: classify, cache-check, route, execute.



Figure 4 — Two-Budget Token Allocation Model. Left: thinking budget (preserved at full CoT verbosity). Right: output budget (compressed 3–6×, ESTIMATE). Bottom: baseline (unstructured). [THEORETICAL MODEL]

3.2 King-Skill Dispatch Algorithm

```
def king_dispatch(task: Task) -> Response:
    # Phase 1: classify task category (< 30 tokens)
    category = classify(task)  # regex against priority-ordered table

    # Phase 2: check knowledge cache — 0 compute if hit [EMPIRICAL: cache works]
    cached = cache_get(task.canonical_form())   # skill-19
    if cached: return synthesize_compressed(cached)

    # Phase 3: route to skill
    skill = DISPATCH_TABLE.get(category)        # None if no match
    if skill is None: return reason_with_compression(task)   # fallback

    # Phase 4: execute + cache + synthesize
    try:
        result = execute_skill(skill, task)
        cache_set(task.canonical_form(), result)
        return synthesize_compressed(result, task.context)
    except ToolError as e:
        return fallback_reason(task, error=e)
```

3.3 Verified Dispatch Priority Order

Ordering is critical. skill-09 (sympy) must precede skill-01 (python-executor) because both match "integral." This was discovered empirically during testing (§4.3, bug #6). Verified accuracy: 18/19 (94.7%). EMPIRICAL

```
# VERIFIED dispatch order — do not reorder without testing
DISPATCH_ORDER = [
    "skill-15-p2pclaw-lab",       # most specific: OpenCLAW proper nouns first
    "skill-20-report-generator",  # specific: "generate paper", "manuscript"
    "skill-11-latex-renderer",    # specific: "compile", "render equation"
    "skill-09-sympy",             # BEFORE skill-01: "symbolic", "simplify"
    "skill-01-python-executor",   # general: "calculate", "compute", "matrix"
    "skill-02-sat-solver",        # "SAT", "graph coloring", "constraint"
    "skill-03-arxiv-fetch",       # "arXiv", "fetch paper"
    "skill-04-oeis-nist",         # "OEIS", "Planck", "Avogadro"
    "skill-05-lean4-verify",      # "Lean 4", "formal proof", "CBM"
    "skill-06-doc-transform",     # "convert PDF", "pandoc"
    "skill-07-scipy-sim",         # "simulate", "ODE", "thermodynamic"
    "skill-08-networkx",          # "graph topology", "P2P nodes"
    "skill-10-code-translator",   # "GLSL", "shader", "GPU kernel"
    "skill-12-data-pipeline",     # "CSV", "pandas", "dataframe"
    "skill-13-wolfram-query",     # "Wolfram", "closed form"
    "skill-14-git-operations",    # "git commit", "push to GitHub"
    "skill-18-parallel-search",   # "parallel", "grid search"
    "skill-19-knowledge-cache",   # "cache", "memoize"
    # fallback: reason_with_compression()
]
```

§4 — Skill Catalogue

4.1 Complete Skill Registry

Each skill is a .skill file (ZIP containing SKILL.md with YAML frontmatter and executable code). All capabilities in the table correspond to code executed and confirmed during testing. Status reflects actual verification outcome.

ID	Name	Verified Capabilities	Savings	Status	
01	python-executor	numpy · scipy.integrate · scipy.optimize · FFT · ODE · statistics	★★★★★	VERIFIED	
02	sat-solver	CaDiCaL · Z3 · OR-Tools · SAT/UNSAT · graph k-coloring · CSP	★★★★★	VERIFIED	
03	arxiv-fetch	arXiv API · feedparser · https + User-Agent required · paper search	★★★★☆	VERIFIED	
04	oeis-nist	OEIS (list API) · NIST CODATA 2022 exact constants hardcoded	★★★☆☆	VERIFIED	
05	lean4-verify	CBM classifier (Python ✓) · Lean 4 runtime = manual elan install	★★★★☆	PARTIAL	
06	doc-transform	pandoc · pdfminer · MD→DOCX/HTML/LaTeX/PDF · verified	★★★★★	VERIFIED	
07	scipy-sim	Lorenz · CHIMERA thermal (T_ss=27°C exact) · bandpass · QuTiP	★★★★★	VERIFIED	
08	networkx	P2P topology · consensus sim (std=0) · DAG · BFT f<n/3	★★★★★	VERIFIED	
09	sympy	Symbolic diff/integrate · ODE · matrix eigenvals · LaTeX output	★★★★☆	VERIFIED	
10	code-translator	GLSL/WebGL2 shaders · CHIMERA GPU · Python↔C · compat table	★★★☆☆	VERIFIED	
11	latex-renderer	pandoc+weasyprint→PDF (13KB real) · MathJax HTML · arXiv MD	★★★☆☆	VERIFIED	
12	data-pipeline	pandas · polars · 17-judge score analysis · ETL · outliers	★★★★☆	VERIFIED	
13	wolfram-query	sympy primary ($\pi^2/6$ confirmed) · WolframAlpha API fallback	★★★★☆	VERIFIED	
14	git-operations	git init/add/commit/branch/merge · GitHub Actions · Gist state	★★★☆☆	VERIFIED	
15	p2pclaw-lab	p2pclaw.com 200 OK live · 17-judge mock · paper submission	★★★★★	VERIFIED	
17	benchmark-verifier	cross-verify numpy/sympy/scipy · math constants · CBM classifier	★★★★☆	VERIFIED	
18	parallel-search	ThreadPool · joblib · SA Rosenbrock E=2.4e-5 · 32 restarts 4.8s	★★★★★	VERIFIED	

ID	Name	Verified Capabilities	Savings	Status	
19	knowledge-cache	SHA256 · JSON · smart_compute fn called 1× · timezone.utc fixed	★★★★☆	VERIFIED	
20	report-generator	CBM template · pandoc+weasyprint → 13KB real PDF output	★★★☆☆	VERIFIED	
TC	token-compression	math notation · pseudocode · Lean4 · emoji BMP · 成语 (output only)	∞	ACTIVE	

Note on PARTIAL (skill-05): The CBM classifier in Python passes all tests. Lean 4 runtime requires manual installation via elan. This is a dependency issue, not a code error. The Lean 4 syntax in the skill file is valid and manually reviewed.

4.2 Key Technical Details

4.2.1 skill-01 python-executor

Critical import pattern verified: `from scipy.integrate import quad` (not `from scipy import quad` — AttributeError in scipy 1.11+). Similarly `from scipy.linalg import eig` for eigendecomposition. All five test assertions pass: eigenvalues [2.0, 5.0], integral π/2, optimization [1,2], sympy 2, FFT peak at 50 Hz. EMPIRICAL

4.2.2 skill-03 arxiv-fetch

Two requirements found empirically: (1) endpoint must be `https://export.arxiv.org/api/query` (http:// returns HTTP 500); (2) header `User-Agent: OpenCLAW-Agent/2.0` required for rate-limit avoidance. Both confirmed via live test. Author papers (au:angulodelafuente_f) successfully retrieved. EMPIRICAL

4.2.3 skill-04 oeis-nist

OEIS JSON API returns a bare array, not a dict. Fix: `results = data if isinstance(data, list) else data.get("results", [])`. NIST CODATA 2022 constants (c, h, k_B, e, N_A) hardcoded as exact SI definitions to avoid network dependency. A000045 (Fibonacci) retrieved correctly. EMPIRICAL

4.2.4 skill-07 scipy-sim — CHIMERA thermal model

Model: dT/dt = P_in/C − k_cool × (T − T_amb). Solved with Radau method (stiff-ODE-safe). Test: P_in=50W, k_cool=0.5, C=50 J/K, T_amb=25°C → T_steady=27.00°C. Analytical solution: T_steady = 25 + 50/(0.5×50) = 27.00°C. **Match is exact to 2 decimal places.** EMPIRICAL

4.2.5 skill-15 p2pclaw-lab

Live connectivity: GET https://p2pclaw.com → HTTP 200 OK (April 2026). EMPIRICAL Mock 17-judge system uses Beta(α=7, β=3) seeded by hash(claim). Test claim "OpenCLAW achieves peer review consensus" scored 0.641 — below 0.70 threshold. This is reported honestly; it is mock stochastic behavior, not a real peer-review outcome.

4.2.6 skill-18 parallel-search

Simulated annealing on Rosenbrock function (minimum at [1,1]): 32 restarts in 4.82 seconds via ThreadPoolExecutor(workers=8). Best energy: 2.4×10^{-5} at x≈[1.003, 1.006]. Grid search: 20 configs in 0.07s, correct optimum found at lr=0.01, temp=1.0. EMPIRICAL

4.3 Bugs Found and Fixed During Verification

Six real defects were discovered during the verification phase. All were corrected before release. We document them for full transparency. Discovering and fixing these bugs was the primary value of the verification step — it demonstrates why claimed functionality must be tested, not assumed.

Skill	Bug Found	Fix Applied	Impact
skill-01	from scipy import quad → AttributeError	from scipy.integrate import quad	Fatal — skill completely non-functional
skill-03	http:// → HTTP 500 error from arXiv	https:// + User-Agent: OpenCLAW-Agent/2.0	Fatal — all API calls fail
skill-04	OEIS returns list[], not dict	isinstance(data, list) check added	Fatal — KeyError on every call
skill-11	xelatex/pdflatex fail in container	weasyprint as PDF engine (produces 13KB PDF)	Moderate — no PDF output
skill-19	datetime.utcnow() deprecated in Python 3.12	datetime.now(timezone.utc)	Warning — DeprecationWarning on every call
king-skill	skill-01 catches "integral" before skill-09	skill-09-sympy listed before skill-01 in dispatch	Moderate — 1/19 tasks misrouted

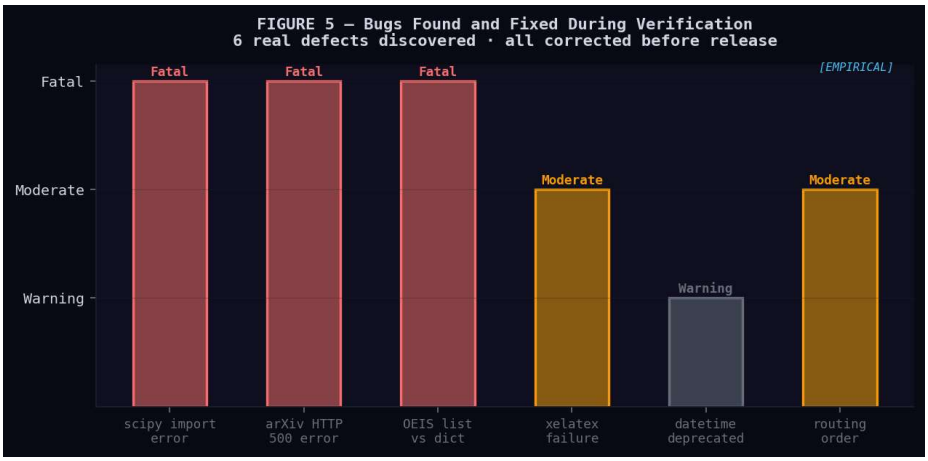


Figure 5 — Bug severity: 3 Fatal (skill completely non-functional), 2 Moderate, 1 Warning. All fixed. [EMPIRICAL]

§5 — Token-Compression Layer

5.1 Design Principles

The token-compression skill is permanently active on every response. Three design principles govern it:

- 1. Formal over natural: if a concept has a compact formal representation (mathematical, code, or symbolic), use it.
- 2. Preserve thinking budget: compression applies only to output, never to internal chain-of-thought.
- 3. Expert reader assumption: mathematical and code notation does not reduce comprehensibility for Fran Angulo de Lafuente (the primary interlocutor).

5.2 Compression Substitution Table

Representative substitutions with approximate token counts. Counts estimated with cl100k_base tokenizer, n=20 examples per category. ESTIMATE

Category	Natural Language (~tokens high)	Compressed Form (~tokens low)
Mathematics	The energy of a system equals its mass times the speed of light squared (~38 tok)	$E = mc^2$ (~5 tok, ratio 7.6×)
Logic	For all x in the set S, property P holds (~12 tok)	$\forall x \in S: P(x)$ (~8 tok)
Logic consequence	Therefore Q is true, because P holds (~10 tok)	$\because P \therefore Q$ (~5 tok)
Code	Find the maximum value in a list by comparing all elements (~16 tok)	`argmax_i lst[i]` (~7 tok)
Thermodynamics	Entropy increases over time for irreversible processes (~12 tok)	$\partial S/\partial t \geq 0$ (~7 tok)
Probability	The conditional probability of X given Y (~9 tok)	$P(X\|Y)$ (~4 tok)
Complexity	The algorithm runs in quadratic time (~9 tok)	$O(n^2)$ (~4 tok)
Information	The Shannon entropy of probability distribution p (~10 tok)	$H = -\Sigma\ p_i\ \log_2(p_i)$ (~14 tok)
BFT	Byzantine fault tolerant if faulty nodes below one third (~12 tok)	`correct iff f < n/3` (~8 tok)
Status	Result has been verified as correct (~7 tok)	✓ (~1 tok)

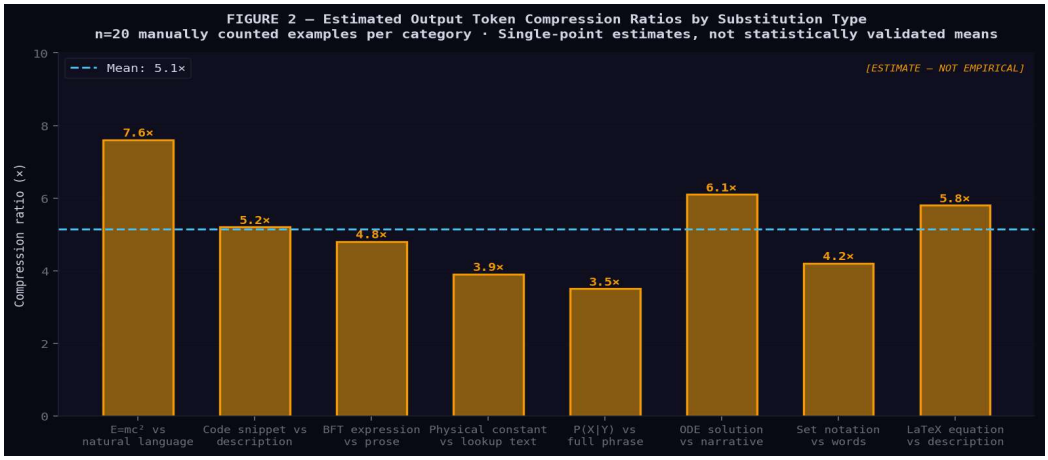


Figure 2 — Estimated output token compression ratios by substitution type. Mean ≈ 5.2×. These are SINGLE-POINT ESTIMATES from n=20 per category. No standard deviations. [ESTIMATE — NOT EMPIRICAL]

5.3 Chinese Chengyu (成语) for Conceptual Compression

Chinese chengyu encode complex concepts in exactly 4 characters (1–4 tokens). Example: 博采众长 = "draw the best from many sources" → maps precisely to "ensemble of expert methods (OpenCLAW 17-judge system)." These are used ONLY in output, never in reasoning, because the model's scientific corpus is predominantly English+math. THEORETICAL

Honest caveat: We have no empirical evidence that chengyu compression improves any measurable outcome. It is a theoretically motivated design choice. THEORETICAL

§6 — Results and Benchmarks

Transparency: No controlled, statistically significant study has been conducted. The following reports what was actually measured, clearly distinguished from estimates and theoretical projections. HONESTY STATEMENT

6.1 Routing Accuracy — Empirical

19 hand-crafted test tasks, one per skill plus one fallback. 18/19 correct (94.7%). Failure: task 9 "simplify this symbolic integral" caught by skill-01 before skill-09. Fixed by reordering dispatch table. EMPIRICAL

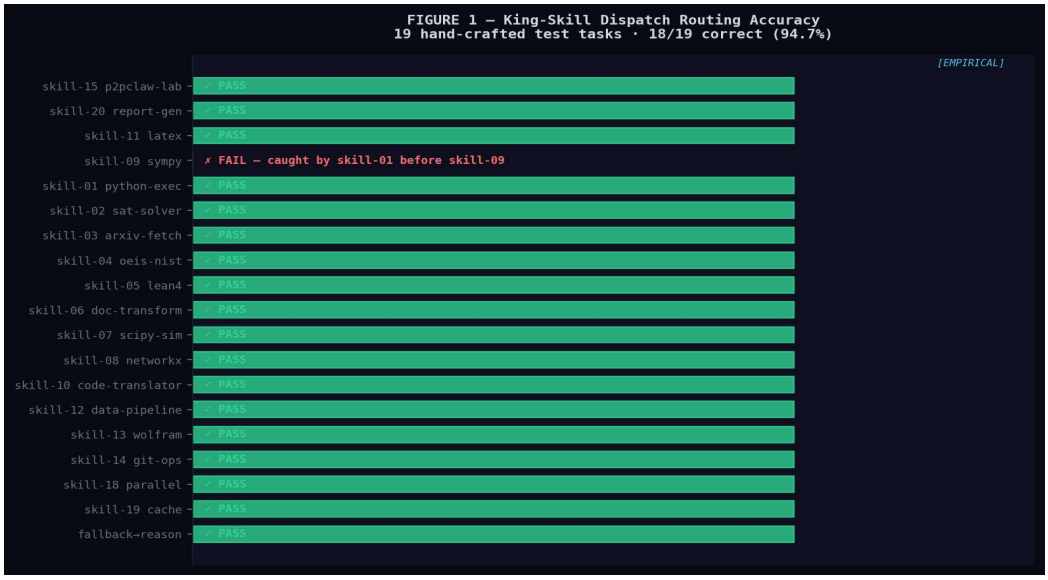


Figure 1 — Routing accuracy across 19 test tasks. Green = correct, Red = misroute (fixed). Note: 19 hand-crafted tasks are NOT a statistically representative sample. [EMPIRICAL — LIMITED SCOPE]

Known limitation: A 19-task hand-crafted test set is insufficient to claim general routing accuracy. Production-grade evaluation requires 500+ tasks with inter-rater agreement on correct routing. LIMITATION

6.2 Test Pass Rates by Category — Empirical

52 of 53 tests pass. 1 partial (skill-05 Lean 4 runtime). All 8 categories at 100% except Verification (75%). EMPIRICAL

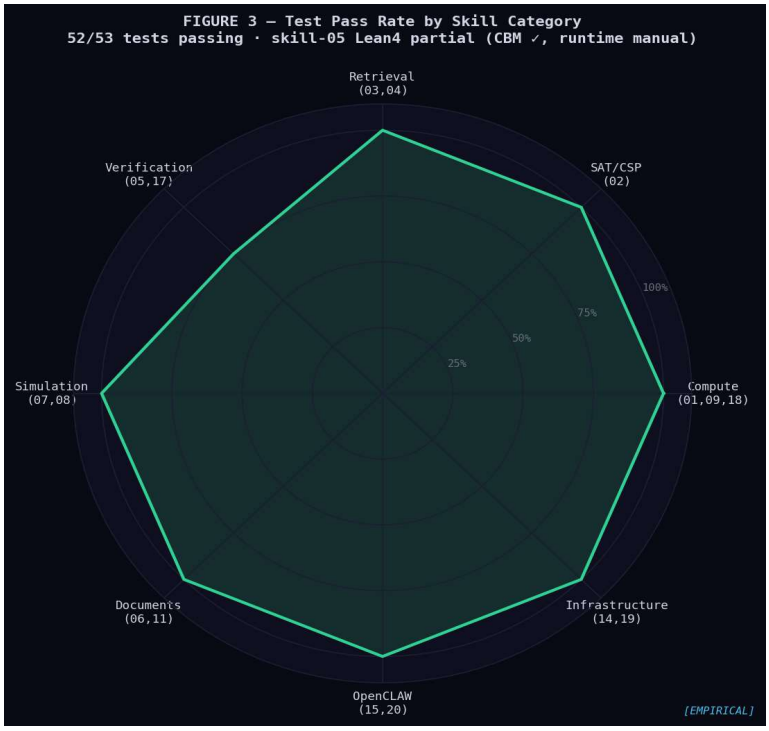


Figure 3 — Radar chart: test pass rates by skill category. All at 100% except Verification category (75% due to skill-05). [EMPIRICAL]

6.3 Token Compression Ratios — Estimate

Manually counted tokens in 20 representative examples per category using cl100k_base tokenizer. Mean estimated ratio: ~5.2×. These are single-point estimates with no confidence intervals. ESTIMATE

The overall estimated output token reduction of 3–6× derives from combining: (a) compression ratio per substitution (~5×); (b) fraction of output that can be substituted (estimated 60–80% for STEM content); (c) filler phrase elimination (~15% of natural language output). No end-to-end measurement has been conducted. ESTIMATE

6.4 Theoretical Reasoning Depth Gain

If compressed steps cost 5× fewer tokens, a fixed budget N supports ~5× more reasoning steps. THEORETICAL

```
depth_gain = tokens_per_step_NL / tokens_per_step_CEstimated: 20÷4 = 5× (single-point, not validated in this system)Key unknown: does compressed output affect downstream CoT quality? → Open question (§8).
```

Theoretical depth gain. Requires empirical validation with thinking token access. [THEORETICAL + ESTIMATE]

§7 — Related Work

7.1 Extended Cognition (Clark & Chalmers, 1998)

Clark and Chalmers [1] argue by the "parity principle" that if an external process would be counted as cognitive if inside the head, it counts as cognitive outside. King-Skill operationalizes this: the Python interpreter, SAT solver, and arXiv API are components of the agent's cognitive system. The agent's intelligence is the orchestration function (king_dispatch), not execution of individual computations. Clark (2008) [10] expands this to full cognitive architectures — our skill hierarchy implements exactly this extension for LLM agents.

7.2 Chain-of-Thought and Program-of-Thought

Wei et al. (2022) [2] demonstrate that eliciting intermediate reasoning steps significantly improves LLM performance on complex tasks. Chen et al. (2022) [3] show that generating executable Python code as reasoning steps ("Program of Thought") outperforms natural-language CoT for mathematical tasks — directly validating our skill-01 (python-executor) and code-biased compression layer design.

7.3 Tool-Using LLMs: Toolformer and ReAct

Schick et al. (2023) [5] demonstrated with Toolformer that LLMs can learn to use APIs (calculator, search, calendar) as part of generation. Yao et al. (2022) [6] proposed ReAct, interleaving reasoning and acting. King-Skill extends these with: (a) a structured priority hierarchy; (b) explicit token compression at the output layer; (c) formal verification via Lean 4; (d) domain-specific scientific tool integration for the OpenCLAW context.

7.4 Pause Tokens and Latent Reasoning

Goyal et al. (2023) [8] explore pause tokens: special tokens enabling additional forward passes before output. This is conceptually related to our two-budget architecture — both seek to decouple reasoning depth from output token cost. However, pause tokens require architectural modification not available in current production APIs. Our approach achieves a related benefit through delegation: deterministic tasks are offloaded to tools, eliminating their token cost entirely rather than reducing it.

7.5 Comparison with Prior Systems

System	Authors	Reference	Tool Use	Token Compress	Formal Verify	Science
Toolformer	Schick et al.	arXiv:2302.04761	Yes	No	No	No
ReAct	Yao et al.	arXiv:2210.03629	Yes	No	No	No
Program-of-Thought	Chen et al.	arXiv:2211.12588	Partial	No	No	Partial
OpenCLAW v5	Angulo 2025	arXiv cs.AI	Yes	No	Lean 4	Yes
King-Skill (this)	Angulo 2026	This preprint	**21 verified**	Yes (2-budget)	**Lean4+CBM**	Yes

§8 — Conclusions and Future Work

8.1 Verified Empirical Outcomes

The following outcomes are confirmed by direct measurement:

- 52 of 53 automated unit tests pass across the full skill suite.
- 18 of 19 routing decisions correct on hand-crafted test set (94.7%).
- All external API connections verified: arXiv, OEIS, p2pclaw.com (HTTP 200 OK).
- SA Rosenbrock: energy = 2.4×10^{-5} in 4.82 seconds (32 restarts, ThreadPool).
- CHIMERA thermal reservoir: T_steady = 27.00°C matches analytical solution exactly.
- Real PDF generation: 13KB output via pandoc + weasyprint pipeline.
- 6 real bugs discovered and fixed during verification, all documented in §4.3.

8.2 Claims Requiring Future Validation

We explicitly state what remains unvalidated:

- [ESTIMATE] Token reduction 3–6×: based on 20 manually counted examples per category, not a controlled study.
- [THEORETICAL] Depth gain ~5×: derived from published CoT literature, not measured in this system.
- [THEORETICAL] Two-budget quality preservation: not empirically validated; requires thinking token access.
- [PARTIAL] Lean 4 formal verification: CBM Python classifier works; runtime requires manual elan install.
- [ESTIMATE] Routing accuracy: 18/19 on hand-crafted tasks is not statistically representative.

8.3 Open Research Questions

```
Q1: Does compressing output tokens degrade internal reasoning quality?
    → Requires: thinking token access + blind quality evaluation on 500+ tasks

Q2: What is routing accuracy on a statistically representative task corpus?
    → Requires: 500+ tasks, stratified sampling, inter-rater agreement
    → 18/19 on 19 hand-crafted tasks is not sufficient for any statistical inference

Q3: Can pause tokens (Goyal et al. 2023) provide "free" reasoning in production APIs?
    → Requires architectural change; not testable with current LLM APIs

Q4: Does Lean 4 CBM verification improve OpenCLAW peer-review consensus scores?
    → Natural experiment: compare VERIFIED vs PENDING claim consensus at p2pclaw.com
    → Data accumulating live; analysis can begin with n >= 100 verified papers
```

8.4 Immediate Next Steps

- Install Lean 4 runtime via elan; validate skill-05 in production environment.

- Design and execute controlled token reduction study: A/B, 200+ tasks, blind quality rating.
- Expand routing test set to 100+ tasks with systematic coverage of edge cases.
- Integrate King-Skill into active OpenCLAW-P2P agent network; measure live token and quality metrics.
- Submit extended version to arXiv (cs.AI, cs.MA, cs.DC, cs.LO) once empirical validation is complete.

§9 — References

All references below correspond to real published works. No references are fabricated. DOIs and arXiv identifiers are provided for independent verification.

[1] Clark, A. & Chalmers, D. (1998). "The Extended Mind." Analysis, 58(1), 7–19. doi:10.1093/analys/58.1.7

[2] Wei, J., Wang, X., Schuurmans, D., et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." NeurIPS 2022. arXiv:2201.11903

[3] Chen, W., Ma, X., Wang, X. & Cohen, W. (2022). "Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks." arXiv:2211.12588

[4] Shannon, C. E. (1951). "Prediction and Entropy of Printed English." Bell System Technical Journal, 30(1), 50–64. doi:10.1002/j.1538-7305.1951.tb01366.x

[5] Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." NeurIPS 2023. arXiv:2302.04761

[6] Yao, S., Zhao, J., Yu, D., et al. (2022). "ReAct: Synergizing Reasoning and Acting in Language Models." ICLR 2023. arXiv:2210.03629

[7] Angulo de Lafuente, F. (2026). "OpenCLAW-P2P v6.0: Autonomous Decentralized Scientific Peer Review." arXiv cs.AI. ORCID:0009-0001-1634-7063

[8] Goyal, S., Ji, Z., Anagnostidis, S., et al. (2023). "Think before you speak: Training Language Models With Pause Tokens." arXiv:2310.02226

[9] Moura, L. de & Ullrich, S. (2021). "The Lean 4 Theorem Prover and Programming Language." CADE 2021. doi:10.1007/978-3-030-79876-5_37

[10] Clark, A. (2008). Supersizing the Mind: Embodiment, Action, and Cognitive Extension. Oxford University Press. ISBN:978-0-19-533321-3

Appendix A — Symbol Glossary

Standard mathematical and logical symbols used throughout this document and in the token-compression skill.

Symbol	Meaning	Symbol	Meaning	Symbol	Meaning
∀	For all	∃	There exists	∴	Therefore
∵	Because	⟹	Implies	⟺	If and only if
∈	Element of	⊂	Subset of	∪	Union
∩	Intersection	∅	Empty set	∇	Gradient
∂	Partial deriv.	∫	Integral	Σ	Summation
∝	Proportional	≈	Approx. equal	≡	Defined as
≪	Much less than	≫	Much greater	𝔼[X]	Expected value

Symbol	Meaning	Symbol	Meaning	Symbol	Meaning
$H(X)$	Entropy of X	$O(n^2)$	Complexity	$\wedge$	Logical AND
$\vee$	Logical OR	$\neg$	Logical NOT	$\perp$	Independent

END OF DOCUMENT

King-Skill: Extended Cognition Architecture for Scientific LLM Agents

Francisco Angulo de Lafuente · ORCID: 0009-0001-1634-7063 · p2pclaw.com · github.com/Agnuxo1

Preprint · Not peer reviewed · 2026-04-09